Practice Set: Function Overloading in C++

Section 1: Conceptual & Practical Questions (1–10)

Question 1: Define **Polymorphism** in Object-Oriented Programming and outline its classification hierarchy in C++ (Compile-Time vs. Run-Time) along with their specific implementations.

Question 2: What is **Function Overloading** in C++? Explain the practical advantage of using a single overloaded function name over inventing artificial identifiers like addInt() and addDouble().

Question 3: Explain how function overloading works in memory. Describe the mechanism of **Name Mangling (Name Decoration)** and how distinct mangled symbols and RAM addresses are assigned to overloaded functions.

Question 4: Detail the **three valid criteria** for overloading a function in C++. Provide a valid prototype signature pair illustrating each criterion.

Question 5: State the critical rule regarding function **Return Types** in function overloading. Why does declaring int add(int, int) and double add(int, int) trigger a compile-time error?

Question 6: Detail the structured **three-step resolution algorithm** executed by the C++ compiler when matching a function call to the best viable overloaded candidate.

Question 7: Compare **Function Overloading** and **Function Overriding** across five key criteria: Polymorphism Type, Scope, Function Signature Requirements, Keywords Required, and Execution Performance.

Question 8: What is an **Ambiguous Call Error** during the overload resolution process? Explain how mixing function overloading with default arguments can introduce ambiguity.

Question 9: Explain the difference between **Type Promotion** (e.g., char to int, float to double) and **Standard Type Conversion** (e.g., int to double) in the compiler's overload resolution hierarchy.

Question 10: Write a complete C++ program that implements an overloaded function add() handling:

Two integers (int, int)

Three integers (int, int, int)

Two double-precision values (double, double)

Mixed types (int, double and double, int)

Section 2: Interview & Viva Questions (11–20)

Question 11 (Viva): Does function overloading introduce any runtime performance overhead during program execution?

Question 12 (Viva): What is the role of an object-file linker in relation to overloaded function names?

Question 13 (Viva): Can two functions be overloaded if their parameter names differ but their parameter data types and count remain identical?

Question 14 (Viva): What happens if an exact type match is not found for an overloaded call, but a float argument can be promoted to double?

Question 15 (Viva): Why does the compiler reject void compute(int x) and void compute(int x, int y = 0) when invoked as compute(10);?

Question 16 (Viva): In what memory segment do the distinct mangled function routines reside during execution?

Question 17 (Viva): Which keyword is mandatory in a base class to implement Run-Time Polymorphism (Function Overriding) instead of Compile-Time Polymorphism?

Question 18 (Viva): Is function overloading classified as Early Binding or Late Binding?

Question 19 (Viva): What type of compilation error is generated if the compiler finds multiple equally viable candidates through standard type conversion?

Question 20 (Viva): How does the compiler differentiate between add(int, double) and add(double, int)?

Answers & Explanations

Answer: 01

Polymorphism: Derived from the Greek words *poly* (many) and *morph* (forms), polymorphism refers to the ability of an entity (such as a function or operator) to exhibit different behaviors depending on data types or runtime context.

Classification Hierarchy:

Compile-Time (Static / Early Binding): Resolved during compilation. Includes **Function Overloading** and **Operator Overloading**.

Run-Time (Dynamic / Late Binding): Resolved during program execution. Includes **Function Overriding** (Virtual Functions) via inheritance.

Answer: 02

Function Overloading: A mechanism where two or more functions in the same scope share the exact same function name but possess different parameter lists.

Advantage: It provides a clean, unified interface. Instead of forcing programmers to memorize artificial, type-specific identifiers (e.g., addInt(), addDouble(), addThreeInts()), the caller simply invokes add() and allows the compiler to bind the appropriate implementation based on the supplied arguments.

Answer: 03

Mechanism: Linkers and CPUs require unique memory entry points for every routine. The C++ compiler resolves overloaded functions using **Name Mangling (Name Decoration)**.

Process: The compiler encodes parameter types and counts directly into the function name to generate a unique internal assembly symbol (e.g., _Z3addii for add(int, int) and _Z3adddd for add(double, double)).

Memory Assignment: Each distinct mangled symbol is assigned its own discrete address in the code/text memory segment, ensuring zero runtime lookup overhead.

Answer: 04

Two or more functions can be overloaded if their parameter signatures differ in:

Number of Parameters: e.g., int add(int, int) vs. int add(int, int, int).

Data Types of Parameters: e.g., int add(int, int) vs. double add(double, double).

Sequence / Order of Data Types: e.g., double add(int, double) vs. double add(double, int).

Answer: 05

Return Type Rule: Functions **cannot** be overloaded based solely on differing return types.

Reason: When the compiler parses an invocation statement like add(5, 10);, it matches arguments against parameter types. Because the return value can be ignored or discarded by the caller, the compiler has no way to infer which function to invoke, producing an immediate compilation error.

Answer: 06

The compiler resolves function calls through a 3-step candidate matching algorithm:

Step 1 (Exact Type Match): The compiler searches for a prototype where argument count and parameter types match identically without conversions. If found, it is selected.

Step 2 (Match Through Type Promotion): If no exact match exists, integral and floating-point promotions are applied (char, unsigned char, short ---> int; float ----> double).

Step 3 (Match Through Standard Type Conversion): If promotion fails, standard implicit conversions are attempted (e.g., int $\rightarrow$ double, pointer conversions). If multiple viable candidates match equally well, an ambiguous call error occurs.

Answer: 07

Feature	**Function Overloading**	**Function Overriding**
Polymorphism	Compile-Time (Static / Early	Run-Time (Dynamic / Late

Feature	Function Overloading	Function Overriding
Type	Binding).	Binding).
Scope	Same class or global namespace scope.	Across Base and Derived classes via inheritance.
Function Signature	Must have different parameter lists (count/types).	Must have the exact same signature and return type.
Keywords Required	No special keywords required.	Requires virtual in Base class and override in Derived.
Execution Performance	Faster (Direct compile-time resolution, 0 overhead).	Slight overhead due to Virtual Table (vtable) lookups.

Answer: 08

Ambiguous Call Error: Occurs when the compiler discovers two or more overloaded candidate functions that match a call site with equal priority, making a definitive selection impossible.

Default Arguments Ambiguity: If void compute(int x) and void compute(int x, int y = 0) are defined, invoking compute(10); matches both the single-parameter prototype and the two-parameter prototype with its default value, triggering a compile-time ambiguous call error.

Answer: 09

Type Promotion (Step 2): Strictly widens smaller fundamental types to their standard natural sizes without precision loss (e.g., char/short widen to int, and float widens to double). It takes precedence over standard conversions.

Standard Type Conversion (Step 3): Involves broader conversions that may involve potential truncation or structural conversions (e.g., converting int to double or double to int) and is only evaluated after type promotions fail.

Answer: 10

```
#include <iostream>
using namespace std;
// 1. Two integers
int add(int a, int b)
```

```
{
   cout << "[add(int, int)] -> ";
   return a + b;
}
// 2. Three integers
int add(int a, int b, int c)
{
   cout << "[add(int, int, int)] -> ";
   return a + b + c;
}
// 3. Two doubles
double add(double x, double y)
{
   cout << "[add(double, double)] -> ";
   return x + y;
}
// 4. Mixed types: int, double
double add(int a, double b)
{
   cout << "[add(int, double)] -> ";
   return a + b;
}
// 5. Mixed types: double, int
double add(double a, int b)
{
   cout << "[add(double, int)] -> ";
   return a + b;
}

int main()
{
   cout << add(5, 10) << endl;
   cout << add(5, 10, 15) << endl;
   cout << add(12.5, 7.5) << endl;
   cout << add(15, 10.0) << endl;
   cout << add(0.75, 5) << endl;
   return 0;
}
```

Answer: 11

No. Function overloading is resolved entirely at compile time via static binding and name mangling, resulting in zero runtime performance penalty.

Answer: 12

The linker requires unique symbolic names to bind function call sites to physical code addresses; name mangling provides unique internal symbols so the linker avoids duplicate symbol collisions.

Answer: 13
No. Parameter names are ignored by the compiler during signature checking; only parameter count, types, and sequence determine validity.

Answer: 14
The compiler applies Step 2 (Type Promotion) and promotes the float argument to double, binding it to the candidate that accepts double.

Answer: 15
Because both prototypes can accept a single integer argument, making the call ambiguous at compile time.

Answer: 16
In the **Code / Text Memory Segment**.

Answer: 17
The **virtual** keyword.

Answer: 18
Early Binding (Static Binding / Compile-Time Binding).

Answer: 19
An **Ambiguous Call Error**.

Answer: 20
Through the **Sequence / Order of Data Types** in their parameter signatures.